

Dispatch, multi-act, and injection

Week 5 Lecture 2 | Browser Use: How LLM Agents Drive the Web

Summer 2026

What we will cover

- multi_act: executing several actions per step
- The terminates_sequence abort rule
- Pydantic validation before dispatch
- Parameter injection by name: browser_session, page_extraction_llm, file_system
- How a result feeds the next step
- One step, end to end

multi_act

Several actions per step

`AgentOutput.action` is a **list**.

The agent can emit more than one action per step. The dispatch layer runs them in order.

`max_actions_per_step` (default: **4**) controls the maximum.

"Maximum actions per step, e.g. for form filling the agent can output 4 fields at once."

(browser-use team, 2026, docs: Agent Settings)

The terminates_sequence rule

Some actions change the page and invalidate element indices.

These carry `terminates_sequence=True`: navigate, go back, search.

When a `terminates_sequence` action runs, the remaining queued actions in the step are **skipped**.

```
[input_text(3, "John"), go_to_url("..."), click(5)]  
      ↓                ↓                ↓  
    runs             runs, STOP         skipped
```

Running `click(5)` after `navigate` would interact with the wrong elements.

(browser-use contributors, 2026, [browser_use/tools/registry/views.py](#))

Pydantic validates before dispatch

Before any handler is called, the LLM's action parameters are validated against the `param_model` registered for that action.

If validation fails:

- The action is not called
- The step is marked as an error
- The error counts against `max_failures`

The model cannot invoke a handler with a wrong type or a missing required parameter. Validation happens before any browser interaction.

Parameter injection

Two kinds of parameters

An action handler can have two kinds of parameters:

LLM-supplied: The LLM writes values for these. Example: `index: int, text: str, url: str`. They appear in `param_model` and are included in the action description the LLM reads.

Injected: The dispatch layer supplies these by matching the parameter name to an object in the agent's context. The LLM never sees them.

The injectable names

Name must be exact. Injection is name-based, not type-based.

Parameter name	What is injected
<code>browser_session</code>	The active <code>BrowserSession</code>
<code>page_extraction_llm</code>	The agent's configured LLM
<code>file_system</code>	A <code>FileSystem</code> interface
<code>available_file_paths</code>	<code>list[str]</code> of accessible files
<code>has_sensitive_data</code>	<code>bool</code>
<code>cdp_client</code>	Direct CDP client

"Using incorrect names is the most common cause of tools failing silently."

(browser-use team, 2026, docs: Custom Functions)

Injection example

```
from browser_use import Tools, ActionResult
from browser_use.browser.session import BrowserSession

tools = Tools()

@tools.action(description="Read the current page title")
async def get_page_title(browser_session: BrowserSession) -> ActionResult:
    # browser_session is injected by the dispatch layer.
    # The LLM calls this action with no parameters: {"get_page_title": {}}
    title = await browser_session.get_title()
    return ActionResult(extracted_content=f"Title: {title}")
```

`goal: str` would be LLM-supplied. `browser_session` is injected. One function can have both.

Result to next step

The feedback path

```
ActionResult.extracted_content
    ↓
    collected by dispatch layer
    ↓
    appears in next step's browser-state context
    ↓
    LLM reads it and writes evaluation_previous_goal
    ↓
    ActionResult.long_term_memory → agent's persistent memory
```

`extracted_content` is short-lived context.

`long_term_memory` is designed to survive past the step-history window.

One complete step, annotated

```
PERCEIVE  BrowserSession: URL, title, DOM, selector map      (wk 3)
DECIDE    MessageManager + LLM → AgentOutput / ActionModel  (wk 4)
ACT
  1. read AgentOutput.action (a list)
  2. for each action:
    a. registry lookup by name
    b. Pydantic param validation
    c. inject named dependencies
    d. call handler → ActionResult
    e. if is_done: exit loop
    f. if terminates_sequence: skip remaining
  3. store results → next step's prompt
```

Every action, built-in or custom, passes through this same path.

Take-aways

1. `max_actions_per_step` (default 4) allows several actions per step; the list executes in order.
2. `terminates_sequence=True` on navigate, go back, and search aborts the remaining queued actions because the page changed.
3. Pydantic validates parameters before any handler is called; failed validation counts as a step error.
4. Injection is name-based: `browser_session`, `page_extraction_llm`, `file_system`, `available_file_paths`. A wrong name produces silent failure.
5. `extracted_content` from the result appears in the next step's prompt; `long_term_memory` persists beyond the step window.

"We execute the actions until the page changes."

What is next

- **Section:** Follow one click live through the dispatch layer and observe multi-act aborting a sequence
- **Week 6:** Writing your own actions with `@tools.action`, typed Pydantic parameters, `ActionResult`, and `allowed_domains`
- **HW 3:** Build a custom-tool agent with structured output (due end of week 6)